

# Navi Protocol Audit Report

---

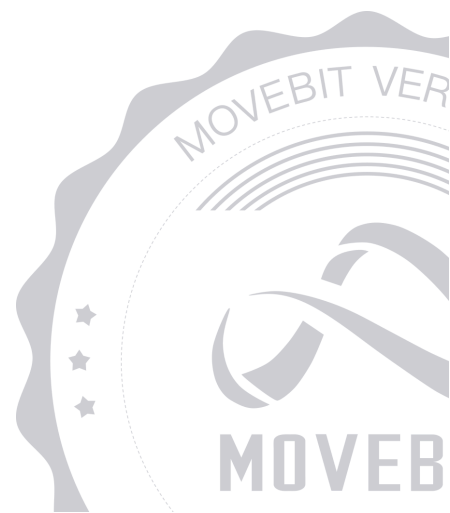


[contact@bitslab.xyz](mailto:contact@bitslab.xyz)



[https://twitter.com/movebit\\_](https://twitter.com/movebit_)

Thu Jul 23 2026



# Navi Protocol Audit Report

---

## 1 Executive Summary

### 1.1 Project Information

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Description | <p>This audit covers the changes introduced in PR #47 of the Navi Protocol Sui repository, corresponding to the diff between commit 762f00aa1d8881a30fb41168015cc815299ce8e1 and commit eab7a4e2dc5133b941b792ae5cc66f494c9e4137. The review focuses primarily on the oracle and lending_core packages, including the migration from the legacy Pyth integration to the pyth_pro_compatible oracle stack, the introduction of updated v2/v3 oracle read and price-update paths, deprecation stubs preserved for upgrade compatibility, related dependency and package-configuration changes, and version bumps that introduce corresponding object-migration requirements.</p> |
| Type        | DeFi                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Auditors    | poetyellow,N0n3,Nebula                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Timeline    | Thu Jul 23 2026 - Thu Jul 23 2026                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Languages   | Move                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Platform    | Sui                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Methods     | Architecture Review, Unit Testing, Manual Review                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Source Code | <a href="https://github.com/naviprotocol/protocol">https://github.com/naviprotocol/protocol</a>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

|         |                                                          |
|---------|----------------------------------------------------------|
| Commits | <a href="#">eab7a4e2dc5133b941b792ae5cc66f494c9e4137</a> |
|---------|----------------------------------------------------------|

## 1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

| ID   | File                                      | SHA-1 Hash                               |
|------|-------------------------------------------|------------------------------------------|
| APY  | oracle/sources/adaptor_pyth.move          | 394c068160906c94ffd2e55d913ee73b8ff8df7b |
| OCO  | oracle/sources/oracle_constants.move      | 03aaccbd8d897e7877f9346908776caeb4640747 |
| OPR1 | oracle/sources/oracle_pro.move            | 1a492adfa97fb36bed45ed3ec49e615e2947981f |
| ODG  | oracle/sources/oracle_dynamic_getter.move | 9313ca0c5576d921d2a4502931a762679c36c347 |
| CON1 | lending_core/sources/constants.move       | c54293a3c7754b877f43c1003436bef9854c2008 |

## 1.3 Issue Statistic

| Item          | Count | Fixed | Acknowledged |
|---------------|-------|-------|--------------|
| Total         | 3     | 0     | 3            |
| Critical      | 0     | 0     | 0            |
| Major         | 0     | 0     | 0            |
| Medium        | 0     | 0     | 0            |
| Minor         | 3     | 0     | 3            |
| Informational | 0     | 0     | 0            |

## 1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

## 1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

### (1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

### (2) Code Review

The code scope is illustrated in section 1.2.

### (3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

### (4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

## 2 Summary

This report has been commissioned by [Navi Protocol](#) to identify any potential issues and vulnerabilities in the source code of the [Navi Protocol](#) smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 3 issues of varying severity, listed below.

| ID    | Title                                                                                                                         | Severity | Status       |
|-------|-------------------------------------------------------------------------------------------------------------------------------|----------|--------------|
| DCA-1 | Division by Zero in <code>dynamic_health_factor</code> May Cause Denial of Service for Empty-Account Preview Flows            | Minor    | Acknowledged |
| DCA-2 | Inconsistent Borrow Rate Preview Formula Above <code>optimal_utilization</code> in <code>dynamic_calculate_borrow_rate</code> | Minor    | Acknowledged |
| OPR-3 | Missing Confidence Interval Validation in Pyth Price Handling                                                                 | Minor    | Acknowledged |

## 3 Participant Process

Here are the relevant actors with their respective abilities within the [Navi Protocol](#) Smart Contract :

Here is a participant process description for the `oracle` , `oracle_manage` , `oracle_pro` , `lending_core` , `manage` , `incentive_v3` , and related modules.

In this system, `Storage` maintains reserve configuration, user collateral and debt accounting, and market-level pause/version state. Per-asset `Pool` objects custody liquidity, while `PriceOracle` stores normalized asset prices and `OracleConfig` defines feed-level routing, freshness, deviation, and range-validation rules across Pyth Pro Compatible, Supra, and Switchboard providers. User-facing lending flows are primarily routed through `incentive_v3` wrappers, while many legacy `lending` and `incentive_v2` entrypoints are retained only for upgrade compatibility and now abort.

### Oracle Admin

- The `OracleAdminCap` holder can initialize and manage shared oracle configuration through `create_config()` .
- The `OracleAdminCap` holder can migrate oracle objects to the current package version through `version_migrate()` , which updates both `OracleConfig` and `PriceOracle` .
- The `OracleAdminCap` holder can pause or resume oracle-config-based pricing through `set_pause()` .
- The `OracleAdminCap` holder can create and configure feeds through `create_price_feed()` , `set_enable_to_price_feed()` , `set_max_timestamp_diff_to_price_feed()` , `set_price_diff_threshold1_to_price_feed()` , `set_price_diff_threshold2_to_price_feed()` , `set_max_duration_within_thresholds_to_price_feed()` , `set_maximum_allowed_span_percentage_to_price_feed()` , `set_maximum_effective_price_to_price_feed()` , `set_minimum_effective_price_to_price_feed()` , and `set_historical_price_ttl_to_price_feed()` .

- The `OracleAdminCap` holder can register and manage provider-specific routing metadata through `create_pyth_oracle_provider_config()` , `create_supra_oracle_provider_config()` , `create_switchboard_oracle_provider_config()` , the corresponding pair-id update functions, and the provider enable/disable functions.
- The `OracleAdminCap` holder can choose the active primary and secondary providers for each feed through `set_primary_oracle_provider()` and `set_secondary_oracle_provider()` .
- The `OracleAdminCap` holder can configure the base `PriceOracle` object through `set_update_interval()` , register supported oracle IDs through `register_token_price()` , and mint additional feeder capabilities through `create_feeder()` .

## Oracle Feeder / Keeper

- An `OracleFeederCap` holder can still update prices directly in `PriceOracle` through `update_token_price()` and `update_token_price_batch()` .
- A keeper or integration can update config-routed prices through `oracle_pro::update_single_price_v3()` , supplying the current provider objects for the configured route, including Pyth Pro Compatible price objects where applicable.
- A caller can retrieve a validated price without mutating state through `oracle_dynamic_getter::get_dynamic_single_price_v2()` .
- The upgraded Pyth flow is accepted only after feed enablement, version checks, freshness checks, pair-id matching, cross-provider deviation checks, and final range/history validation succeed.
- Legacy Pyth functions such as `update_single_price_v2()` and the old `get_dynamic_single_price()` path are retained only for upgrade compatibility and now abort.

## Storage Admin

- The `StorageAdminCap` holder can migrate the shared lending `Storage` object to the current package version through `version_migrate()` .

- The `StorageAdminCap` holder can initialize reserves through `init_reserve()` .
- The `StorageAdminCap` holder can create or configure supporting protocol objects through `create_flash_loan_config_with_storage()` , `create_flash_loan_asset()` , `incentive_v3_version_migrate()` , `create_incentive_v3_with_storage()` , `create_incentive_v3_pool()` , `create_incentive_v3_rule()` , and `mint_borrow_fee_cap()` .
- The `StorageAdminCap` holder can manage protocol-level market features through `create_new_market()` , `set_borrow_weight()` , `remove_borrow_weight()` , and the `eMode` configuration helpers exposed in `manage.move` .
- The `StorageAdminCap` holder can withdraw accrued borrow-fee balances or reward-fund balances through `withdraw_borrow_fee()` and `withdraw_incentive_v3_reward_fund()` .

## Storage Owner / Risk Manager

- The `OwnerCap` holder can pause or resume the lending market through `set_pause()` .
- The `OwnerCap` holder can update reserve risk parameters through functions such as `set_supply_cap()` , `set_borrow_cap()` , `set_ltv()` , `set_treasury_factor()` , `set_base_rate()` , `set_multiplier()` , `set_jump_rate_multiplier()` , `set_reserve_factor()` , `set_optimal_utilization()` , `set_liquidation_ratio()` , `set_liquidation_bonus()` , and `set_liquidation_threshold()` .
- The `StorageOwnerCap` holder can configure protected-liquidation behavior through `set_designated_liquidators()` and `set_protected_liquidation_users()` .
- The `StorageOwnerCap` holder can initialize certain auxiliary fields through `init_fields_batch()` .

## Incentive / Fee Manager

- The `IncentiveOwnerCap` holder can create and fund V3 reward infrastructure through `create_incentive_v3_reward_fund_with_storage()` , `deposit_incentive_v3_reward_fund()` , `create_incentive_v3_with_storage()` , `create_incentive_v3_pool()` , `create_incentive_v3_rule()` ,

`enable_incentive_v3_by_rule_id()` , `disable_incentive_v3_by_rule_id()` ,  
`set_incentive_v3_reward_rate_by_rule_id()` , and  
`set_incentive_v3_max_reward_rate_by_rule_id()` .

- A `BorrowFeeCap` holder can manage global, asset-specific, and user-specific borrow-fee overrides through `set_borrow_fee_rate()` , `set_asset_borrow_fee_rate()` , `set_user_borrow_fee_rate()` , `remove_incentive_v3_asset_borrow_fee_rate()` , and `remove_incentive_v3_user_borrow_fee_rate()` .

## Pool / Treasury Operator

- A `PoolAdminCap` holder can manage SUI pool-manager integration through `init_sui_pool_manager()` , `enable_manage()` , `disable_manage()` , `refresh_stake()` , `set_target_sui_amount()` , `unstake_vsui()` , and `set_validator_weights_vsui()` .
- A `PoolAdminCap` holder can withdraw treasury balances from pools through `withdraw_treasury()` and, for managed SUI pools, `withdraw_vsui_from_treasury()` .
- Direct reserve withdrawal helpers outside the supported v2 path are restricted or deprecated; for example, `withdraw_reserve_balance()` aborts and `withdraw_reserve_balance_v2()` is the active path.

## User / AccountCap Holder

- A user can create an account capability through `create_account()` .
- A user can supply assets through the active V3 wrappers `incentive_v3::entry_deposit()` and `entry_deposit_on_behalf_of_user()` .
- A user can withdraw assets through `entry_withdraw()` or `entry_withdraw_v2()` , with the v2 path supporting SUI pools that may need staking-related preparation.
- A user can borrow through `entry_borrow()` or `entry_borrow_v2()` , with borrow fees applied through the V3 fee schedule.
- A user can repay their own debt through `entry_repay()` or repay on behalf of another user through `entry_repay_on_behalf_of_user()` .

- A user can enter or exit eMode through `enter_emode()` and `exit_emode()` , or the corresponding `AccountCap` -based helpers.
- A user can claim V3 rewards through `claim_reward_entry()` .
- Legacy public entrypoints in `lending.move` and most public entrypoints in `incentive_v2.move` are retained only for backward compatibility and now abort.

## Liquidator

- A liquidator can close unhealthy debt positions through `incentive_v3::entry_liquidation()` or `entry_liquidation_v2()` .
- Liquidation succeeds only if storage state is current, the user is liquidatable under the protocol's health and liquidation-threshold logic, and the selected collateral/debt pair passes reserve and market checks.
- When protected-liquidation rules are configured, liquidation permissions can be further constrained through designated-liquidator mappings maintained by protocol operators.

## Upgrade / Migration Notes

- Existing oracle objects must be migrated before post-upgrade oracle-config-based reads and updates can succeed, because the oracle version requirement increases in this range.
- Existing lending `Storage` objects must also be migrated before normal lending operations can resume, because the lending-core version requirement increases in this range.
- The upgraded oracle path replaces legacy Pyth usage with the `pyth_pro_compatible` integration, while preserving old function names only as aborting compatibility stubs.

## 4 Findings

### DCA-1 Division by Zero in `dynamic_health_factor` May Cause Denial of Service for Empty-Account Preview Flows

**Severity:** Minor

**Status:** Acknowledged

**Code Location:**

`lending_core/sources/dynamic_calculator.move`

**Descriptions:**

In the current code version of branch `feat/oracle-pyth-pro-upgrade` (HEAD: `eab7a4e`), `dynamic_health_factor` in `protocol/lending_core/sources/dynamic_calculator.move` is responsible for estimating a user's health factor under hypothetical deposit, withdrawal, borrow, or repay operations. Before producing the projected health factor, this function calls `dynamic_liquidation_threshold` to derive the dynamic liquidation threshold for the simulated account state.

The issue is that `dynamic_liquidation_threshold` in `protocol/lending_core/sources/dynamic_calculator.move` ends with a direct call to `ray_math::ray_div(collateral_health_value, collateral_value)` without guarding the case where `collateral_value == 0`. At the same time, `get_user_assets` in `protocol/lending_core/sources/storage.move` returns empty collateral and loan vectors for an empty account. As a result, when an empty account enters a pure read preview, a borrow preview, or any other preview path that does not introduce positive collateral value, `collateral_value` can remain `0`. The `ray_div` implementation in `protocol/math/sources/ray_math.move` explicitly aborts when the denominator is zero through `RAY_MATH_DIVISION_BY_ZERO`. This behavior is directly confirmed by the unit test in `protocol/lending_core/tests/logic_flow_tests/known_discrepancy_tests.move`, which expects `dynamic_calculator::dynamic_health_factor` to abort with code `1103` in

`math::ray_math` for an empty account. In contrast, the counterpart implementation in `protocol/lending_core/sources/logic.move` already contains a `collateral_value > 0` guard, and the same test file verifies that `logic::user_health_factor` safely returns `address::max()` for the same empty-account scenario.

Accordingly, this issue does not appear to directly endanger on-chain funds, because the affected path is a projected calculation path rather than the settlement logic itself. However, any frontend, router, monitoring service, or automation bot that directly invokes `lending_core::dynamic_calculator::dynamic_health_factor` may encounter unexpected aborts when evaluating a new wallet or an empty account. This can cause denial of service in preview flows, interrupt risk evaluation, and propagate integration-level failures. The defect also introduces inconsistent empty-account semantics between the dynamic preview logic and the regular logic, which increases integration risk and maintenance complexity.

#### Suggestion:

It is recommended to explicitly handle the `collateral_value == 0` case in `dynamic_liquidation_threshold` and align the empty-account semantics of `lending_core::dynamic_calculator` with those already used in `logic.move`. It is also recommended to define a uniform return convention for empty accounts across all preview-only and read-only risk calculation entry points, so that upstream integrations do not experience denial of service or unexpected state-evaluation failures due to unhandled aborts.

## DCA-2 Inconsistent Borrow Rate Preview Formula Above optimal\_utilization in dynamic\_calculate\_borrow\_rate

Severity: Minor

Status: Acknowledged

Code Location:

`lending_core/sources/dynamic_calculator.move`

Descriptions:

In `feat-oracle-pyth-pro-upgrade/lending_core/sources/calculator.move`, `calculate_borrow_rate` computes the borrow rate that is actually applied and persisted by the protocol. In contrast, `dynamic_calculate_borrow_rate` in `feat-oracle-pyth-pro-upgrade/lending_core/sources/dynamic_calculator.move` is used for what-if estimation and preview of borrow rates, and `feat-oracle-pyth-pro-upgrade/lending_ui/sources/calculate.move` contains a mirrored preview implementation. Accordingly, these paths are expected to remain consistent under the same state, with the only intended difference being whether an estimated state change is incorporated, rather than any divergence in the underlying rate model itself.

In the current implementation, both paths use  $\text{base\_rate} + \text{utilization} * \text{multiplier}$  when utilization is below `optimal_utilization`, so their behavior is aligned in that region. However, once utilization exceeds `optimal_utilization`, the real charging path uses  $\text{base\_rate} + \text{optimal\_utilization} * \text{multiplier} + (\text{utilization} - \text{optimal\_utilization}) * \text{jump\_rate\_multiplier}$ , while the preview path uses  $\text{base\_rate} + \text{utilization} * \text{multiplier} + (\text{utilization} - \text{optimal\_utilization}) * \text{jump\_rate\_multiplier}$ . As a result, the preview path adds an extra  $(\text{utilization} - \text{optimal\_utilization}) * \text{multiplier}$  above the kink and therefore systematically overestimates the borrow rate in the high-utilization region. The behavior is reproduced by `feat-oracle-pyth-pro-upgrade/lending_core/tests/logic_flow_tests/known_discrepancy_tests.move`: under a 90%

utilization scenario with an 80% kink, 5% multiplier, and 109% jump multiplier, the actual charged rate is 14.9%, whereas the previewed rate is 15.4%.

This issue does not modify the borrow rate actually charged by the protocol and therefore would not normally lead to direct asset loss. However, it causes a systematic discrepancy in front-end display, pre-transaction estimation, APY quotation, and user decision-making inputs. In the high-utilization region, users may rely on an overstated previewed borrow cost when evaluating borrowing, repayment, or position adjustment decisions, which reduces quotation accuracy and undermines consistency between displayed values and on-chain behavior.

#### Suggestion:

It is recommended to align the preview formula used above `optimal_utilization` with the actual charging formula so that preview logic reflects only the estimated state transition and does not introduce an additional model deviation. It is also recommended to review the mirrored implementations in both `lending_core` and `lending_ui`, and to add regression tests covering the below-kink, at-kink, and above-kink regions to prevent future formula divergence.

# OPR-3 Missing Confidence Interval Validation in Pyth Price Handling

**Severity:** Minor

**Status:** Acknowledged

**Code Location:**

oracle/sources/oracle\_pro.move

**Descriptions:**

The oracle logic uses the Pyth price as long as the feed matches, the timestamp is fresh, and the final value passes cross-source and historical range checks. However, it does not validate the Pyth confidence interval before accepting the price.

Pyth prices contain both a reported price and a confidence interval. The confidence interval shows how uncertain the reported price is. A large `conf` value means the oracle is less certain about the price, even if the price is recent.

In `oracle::adaptor_pyth`, both `get_price_native_v2()` and `get_price_unsafe_native_v2()` read only the price, exponent, and timestamp from the Pyth price object. The `conf` field is not read or checked.

**oracle/sources/adaptor\_pyth.move**

```
let i64_price = pro_price::get_price(&pyth_price_info);
let i64_expo = pro_price::get_expo(&pyth_price_info);
let timestamp = pro_price::get_timestamp(&pyth_price_info) * 1000;
let price = pro_i64::get_magnitude_if_positive(&i64_price);
let expo = pro_i64::get_magnitude_if_negative(&i64_expo);
```

After that, `oracle::oracle_pro::get_price_from_adaptor_v3()` uses the converted value and returns it to the main oracle flow without any validation on `conf`.

**oracle/sources/oracle\_pro.move**

```
if (provider == provider::pyth_provider()) {  
    let pyth_pair_id = oracle::adaptor_pyth::get_identifier_to_vector_v2(pyth_price_info);  
    assert!(sui::address::from_bytes(pyth_pair_id) == sui::address::from_bytes(pair_id),  
error::pair_not_match());  
    let (price, timestamp) =  
oracle::adaptor_pyth::get_price_unsafe_to_target_decimal_v2(pyth_price_info,  
target_decimal);  
    return (price, timestamp)  
};
```

As a result, the protocol can still accept a Pyth price that has a very wide confidence interval. This means the price may pass the current freshness and range checks while still carrying high uncertainty. During volatile market conditions or degraded oracle conditions, this can reduce the reliability of collateral valuation, borrowing checks, and liquidation decisions.

#### Suggestion:

It is recommended to validate the Pyth confidence interval before accepting a Pyth price for protocol use. The validation should reject prices whose `conf` value is too large relative to the reported price.

As described in the Pyth best practices for confidence intervals, the protocol may either use a conservative price derived from the confidence interval, such as `price - conf` for collateral valuation and `price + conf` for debt valuation, or pause new activity when the `conf / price` ratio exceeds a defined threshold. See the Pyth guidance here:

<https://docs.pyth.network/price-feeds/core/best-practices#confidence-intervals>

# Appendix 1

## Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't pose any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

## Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

# Appendix 2

## Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

